Especificação e Divisão do Projeto Final — WEB 2

Data de Entrega: 01/12
Repositório Oficial (GitHub): https://github.com/andresilva05/game-store

1. Definição do Produto: Loja Virtual de Gift Cards (Estilo Bonoxs)

O sistema será um e-commerce especializado na venda de **Gift Cards e Créditos Digitais** (Steam, PlayStation, Xbox, Riot, Nintendo, etc.).

- **Como funcionará o estoque e entrega dos códigos (PINs):**
- Cada Gift Card terá uma quantidade de estoque disponível. No momento em que o cliente finalizar a compra (Checkout), o sistema validará o estoque, dará baixa nas unidades compradas e gerará automaticamente o **Código PIN de resgate** (ex: XXXX-YYYY-ZZZZ), que ficará disponível na área *"Meus Gift Cards"* do cliente.
- **Atores do Sistema:**
    - **Cliente:** Navega pelo catálogo, filtra por plataforma, adiciona Gift Cards ao carrinho, finaliza a compra e visualiza seus códigos PIN adquiridos.
    - **Administrador:** Gerencia o catálogo de plataformas e Gift Cards (cadastro, edição de preços, ativação/inativação e reposição de estoque).

2. Arquitetura Obrigatória (Critérios de Avaliação do Professor)

Para garantir nota máxima e que todos saibam responder qualquer pergunta na apresentação, o projeto seguirá rigorosamente os padrões dados em aula:

1. **Camada View (JSP):** Páginas protegidas em `/WEB-INF/` utilizando **JSTL** (`<c:forEach>`, `<c:if>`) e **EL** (`${...}`), sem código Java misturado no HTML.
2. **Camada Controller (Servlets):** Tratamento de requisições HTTP seguindo o padrão **PRG (Post-Redirect-Get)**.
3. **Camada de Negócio (Service Layer):** Toda validação e regra de negócio ficará isolada nas classes `Service`, disparando `RegraNegocioException` quando necessário.
4. **Camada de Persistência (DAO + JDBC):** Consultas e atualizações isoladas nos DAOs usando `PreparedStatement` (prevenção de SQL Injection), `DataAccessException` e pool de conexões **HikariCP** (`ConnectionFactory`).
5. **Transação ACID:** Controle transacional explícito (`setAutoCommit(false)`, `commit()`, `rollback()`) no fluxo de compra.

3. Estrutura Base do Banco de Dados (Proposta para Validação)

Para atender aos 4 módulos sem retrabalho e facilitar eventual integração futura com **BD 2**, teremos 5 tabelas principais:

- **`usuario`**: `id`, `nome`, `email`, `senha`, `papel` (`'CLIENTE'` ou `'ADMIN'`).
- **`plataforma`**: `id`, `nome` *(ex: Steam, PSN, Xbox, Riot)*.
- **`produto`** *(Gift Cards)*: `id`, `plataforma_id` (FK), `titulo`, `valor_credito`, `preco_venda`, `estoque`, `ativo`.
- **`pedido`**: `id`, `usuario_id` (FK), `data_hora`, `valor_total`.
- **`item_pedido`**: `id`, `pedido_id` (FK), `produto_id` (FK), `preco_unitario`, `codigo_pin`.

4. Divisão de Tarefas da Equipe (4 Módulos de Ponta a Ponta)

Para ninguém ficar travado esperando o outro e para que **todos pratiquem todas as camadas (DAO + Service + Servlet + JSP)** visando a apresentação ao professor, o sistema foi dividido em 4 módulos independentes.

Escolham qual dos módulos em aberto (**1, 2 ou 3**) cada um prefere assumir:

[EM ABERTO] Módulo 1 — Catálogo & Área Administrativa (Admin)

- **Escopo:** CRUD completo de Plataformas e Gift Cards (cadastrar, listar, editar preço/estoque e inativar).
- **Regras de Negócio (Service):** Não permitir cadastro com preço/estoque negativo; não permitir excluir Gift Card que já possua vendas registradas.
- **O que vai construir:** `PlataformaDAO`, `ProdutoDAO`, `ProdutoService`, Servlets/Commands do Admin e JSPs de gerenciamento.

[EM ABERTO] Módulo 2 — Autenticação, Usuários & Segurança

- **Escopo:** Cadastro de novos clientes, tela de Login, Logout e proteção de rotas privadas.
- **Regras de Negócio (Service):** Não permitir cadastro com e-mail duplicado; validar credenciais de acesso e perfil (`CLIENTE` vs `ADMIN`).
- **O que vai construir:** `UsuarioDAO`, `UsuarioService`, Servlets de Login/Cadastro, `AuthenticationFilter` (`@WebFilter` barrando acesso não autorizado) e `SessionListener` (`@WebListener`).

[EM ABERTO] Módulo 3 — Vitrine da Loja & Carrinho de Compras

- **Escopo:** Página inicial da loja com listagem de Gift Cards ativos, filtro por plataforma/busca e gerenciamento completo do Carrinho de Compras na sessão HTTP (`HttpSession`).
- **Regras de Negócio (Service/Sessão):** Adicionar item ao carrinho, alterar quantidade, remover item, calcular total e impedir adição de quantidade superior ao estoque disponível.
- **O que vai construir:** Classes `Carrinho` e `ItemCarrinho`, Servlets do Carrinho/Vitrine e páginas `index.jsp` e `carrinho.jsp` com JSTL e EL.

[DEFINIDO: ANDRÉ] Módulo 4 — Checkout, Transação ACID & "Meus Gift Cards"

- **Escopo:** Fechamento do pedido a partir do carrinho da sessão, geração dos códigos PIN e página da biblioteca do cliente (*"Meus Gift Cards"*).
- **Regras de Negócio & Transação:** Execução atômica com `setAutoCommit(false)`, `commit()` e `rollback()` (insere o pedido + insere os itens com os PINs gerados + atualiza o estoque dos produtos).
- **O que vai construir:** `PedidoDAO`, `PedidoService` (com controle transacional), Servlets de Checkout/Histórico e JSP de visualização dos PINs comprados.

5. Como Clonar e Trabalhar no Repositório

O repositório já está configurado com **Gradle**, **Jakarta EE 10 (Java 21)** e **CI automatizado no GitHub Actions**. A branch `main` é protegida: todo código deve ser feito em uma branch separada e enviado via **Pull Request (PR)**.

Passos para clonar:

```
bash
git clone https://github.com/andresilva05/game-store.git
cd game-store
```

Observação: Antes de começarmos a codificar as funcionalidades, confirmem qual módulo cada um vai pegar (1, 2 ou 3) e se estão de acordo com as 5 tabelas do banco!